Assignment 4 - CSV and JSON data - C++

2/24/2022

Legg til kommentar 0

2/11/2022 til 5/6/2022

Detaljer

Introduction

- Before you start make sure you have completed Course preparations.
- Each assignment below is a separate directory. When you create each project the directory path will be *ikt103g22v/assignments/solutions/assignment_?_?*.

Note: See the lecture resources for links to C++ libraries needed to read CSV / JSON.

4.1 Read CSV

Create a program that opens and reads three data files with the following format:

customers.csv

```
id,name,address
1,"Knut","Knutveien 3"
2,"Lise","Liseveien 7"
```

products.csv

```
id,name,price
1,"banana",5
2,"apple",10
```

orders.csv

```
id,customerid,productid,amount
1,1,1,2
2,2,1,3
3,1,2,4
```

For testing you can use the data above, or create your own files.

Requirements:

After every row is read the following information must be displayed:

- Every customer.
  Output format: *Customer: <name>, <address>*
- Every product.
  Output format: *Product: <name>, <price>*

- Total amount ordered per product.
  Output format: *<product name> amount: <amount>*
- Total money spent per product.
  Output format: *<product name> gross income: <amount>*
- Total money spent per customer.
  Output format: *<name> money spent: <amount>*

Areas of interest: CSV, list, vector, map

4.2 Read JSON

This choice should open and read the file *students.json* which has the following format:

```
[
  {
    "id": 1,
    "name": "Lisa Larsen",
    "age": 22,
    "attendance": 70
  },
  {
    "id": 2,
    "name": "Knut Knutsen",
    "age": 29,
    "attendance": 50
  }
]
```

Implementation requirements:

- Students must be represented in the program as a class called *Student*
- The *Student* class must be declared in student.h
  You can also use student.cpp if your class has functions.
- Each object in the JSON data must be added to a list of *Student* pointers

Functional requirements

After every student is read the following information must be displayed:

- The youngest student.
  Output format: *Youngest: <name>*
- The oldest student.
  Output format: *Oldest: <name>*
- Average student age (rounded down to nearest integer).
  Output format: *Average age: <average>*
- Every student with an attendance lower than 30.
  Output format: *Bad student: <name>*

Areas of interest: JSON, classes, list

Testing and delivery

see *Local testing and Delivery and Bamboo testing* under Assignments.